TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT REPORT

ON

HUMAN PRESENCE DETECTION USING DISTRIBUTED INFERENCE ON RESOURCE CONSTRAINED EDGE DEVICES

Submitted By:

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Asst. Prof. Suwarna Lingden

May , 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT REPORT

ON

HUMAN PRESENCE DETECTION USING DISTRIBUTED INFERENCE ON RESOURCE CONSTRAINED EDGE DEVICES

Submitted By:

Ishwor Raj Pokhrel	(THA079BEI013)
Pramit Khatri	(THA079BEI020)
Preeti Rijal	(THA079BEI028)
Sampada Ghimire	(THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

May , 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled **"Human Presence Detection Using Distributed Inference On Resource Constrained Edge Devices"** submitted by **Ishwor Raj Pokhrel, Pramit Khatri , Preeti Rijal , Sampada Ghimire** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor
Asst. Prof. Suwarna Lingden
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

External Examiner
Er. Birodh Rijal
Co-Founder and CEO, Integrated ICT Pvt. Ltd.
Jwagal, Kupondole, Lalitpur, Nepal

Head of Department
Asst. Prof. Umesh Kanta Ghimire
Department of Electronics and Computer Engineering
Thapathali Campus, IOE

May , 2026

DECLARATION

We hereby declare that the project entitled, **"Human Presence Detection Using Distributed Inference On Resource Constrained Edge Devices"** for the partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in **Electronics, Communication and Information Engineering** is a bonafide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Ishwor Raj Pokhrel (THA079BEI013)
Pramit Khatri (THA079BEI020)
Preeti Rijal (THA079BEI028)
Sampada Ghimire (THA079BEI036)

Date: May , 2026

COPYRIGHT

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project supervisor, **Asst. Prof. Suwarna Lingden**, for his guidance, invaluable feedback, and constant support for the project. His expertise, experience and encouragement have been pivotal in shaping this project.

We are also deeply thankful to **Asst. Prof. Umesh Kanta Ghimire**, Head of the Department, and the entire Department of Electronics and Computer Engineering for providing the necessary facilities and a supportive academic environment for our study.

Finally, we extend our sincere thanks to all the faculty members, friends, and both direct-indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Ishwor Raj Pokhrel (THA079BEI013)
Pramit Khatri (THA079BEI020)
Preeti Rijal (THA079BEI028)
Sampada Ghimire (THA079BEI036)

ABSTRACT

The proliferation of high-resolution image sensors in surveillance and monitoring applications has exposed a fundamental limitation in conventional edge inference pipelines: the spatial downsampling required to fit high-resolution frames into fixed-dimension neural network inputs systematically destroys the fine-grained detail necessary for reliable detection of small or distant subjects. This work presents a distributed tile-based inference architecture for real-time Human Presence Detection (HPD) on commodity microcontroller hardware, designed to preserve spatial resolution while remaining within the memory and compute constraints of resource-limited edge nodes. End-to-end benchmarking reveals that frame capture and tiling completes in 4 Milliseconds (ms), aggregator classification in 0.342 ms, and feature transmission per tile in 472 ms. The dominant bottleneck is tile-level inference on the ESP32-S3, which requires 2,657ms per tile due to the mandatory use of OCTAL PSRAM for the 451KB TFLite arena, yielding a total critical-path latency of approximately 18.9 seconds against a 1,000 ms target frame budget. All three deployed models were verified to be fully INT8 with zero float32 tensors remaining. The results confirm architectural correctness across all pipeline stages while clearly identifying PSRAM-bound inference latency and sequential tile fragmentation as the primary targets for optimisation in future work.

Keywords: *Human presence detection, distributed tile-based inference, local binary convolutional neural network, attention mechanism, TinyML, post-training quantization, ESP32, edge computing.*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the Curve
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
COCO	Common Objects in Context
DAT	Distributed Adaptive Tiling
ERF	Effective Receptive Field
ESP	Espressif Systems
ESP-DL	Espressif Deep Learning library
ESP-NOW	Espressif Non-Beaconing Wireless
FLOPs	Floating Point Operations
FN	False Negative
FP	False Positive
FP32	32-bit Floating Point
FPS	Frames Per Second
GAP	Global Average Pooling
GPU	Graphics Processing Unit
GSD	Ground Sampling Distance
HPD	Human Presence Detection
HVAC	Heating, Ventilation, and Air Conditioning
I2C	Inter-Integrated Circuit
INT4	4-bit Integer
INT8	8-bit Integer
IoT	Internet of Things
IoU	Intersection over Union
IR	Infrared

KB	Kilobyte
LBC	Local Binary Convolutions
LBCNN	Local Binary Convolutional Neural Network
LBP	Local Binary Patterns
LCD	Liquid Crystal Display
LMMs	Large Multimodal Models
mac	Multiply-Accumulate Operations
mAP	Mean Average Precision
MB	Megabyte
MCU	Microcontroller Unit
ms	Milliseconds
OD	Object Detection
OOM	Out of Memory
PCs	Personal Computers
PIR	Passive Infrared
PTQ	Post-Training Quantization
Q-CNN	Quantized Convolutional Neural Network
QAT	Quantization-Aware Training
R-CNN	Regions with Convolutional Neural Network
SAHI	Slicing Aided Hyper Inference
SL	Split Learning
SOD	Small Object Detection
SR	Squeeze Ratio
SRAM	Static Random-Access Memory
TFML	TensorFlow Lite for Microcontrollers
TinyML	Tiny Machine Learning
TP	True Positive

UAV	Unmanned Aerial Vehicle
UHD	Ultra-high Definition
WSN	Wireless Sensor Network
YOLO	You Only Look Once

1. INTRODUCTION

While Convolutional Neural Networks (CNNs) have mastered object detection in controlled environments, moving to real-world, high-resolution processing is still hampered by significant hardware constraints. In many scenarios, simply shrinking a high-definition image to fit into a small processor causes the loss of critical spatial details. By borrowing the tiling methodologies used in high-precision agriculture where massive fields are analyzed piece by piece and combining them with distributed edge computing, this project proposes a scalable solution for real-time HPD. We chose HPD as our primary focus because detecting people in large, open spaces requires the high-level granularity that traditional downsampling destroys. By splitting a high-resolution frame into smaller tiles and distributing the workload across a network of resource-constrained Microcontroller Units (MCUs), we maintain the precision needed to identify human subjects without the need for expensive, centralized hardware. This approach not only ensures accuracy in expansive environments but also keeps data processing local, addressing the latency and privacy concerns essential to modern monitoring systems.

1.1. Background

Standard CNN architectures like You Only Look Once (YOLO) or Faster Regions with Convolutional Neural Network (R-CNN) typically use an input resolution of 416×416 or 640×640. In high-resolution domains like surveillance or precision agriculture, an image can be 4000×3000 pixels.

The Problem of Vanishing Detail

In high-resolution surveillance tasks, the standard practice of resizing input images to fit fixed-size CNN architectures (e.g., 224×224 or 640×640) introduces a critical bottleneck known as Spatial Information Attrition. When an Ultra-high Definition (UHD) frame of 4000-pixel width is downsampled to 400 pixels, the transformation results in a 90% reduction in linear resolution and a 99% loss of total pixel data.

This geometric reduction disproportionately affects small-scale objects. For instance, a human subject occupying a 40×80 pixel area in the original frame is reduced to a mere 4×8 pixel fragment. Such a representation falls significantly below the minimum Effective Receptive Field (ERF) required for a CNN to extract discriminative features like limb orientation or torso contours [1]. At this scale, the high-frequency spatial components that define humanness are discarded, leaving the network to attempt classification based on aliased, low-frequency blobs that are indistinguishable from environmental noise.

Lessons from High-Precision Agriculture

Solution to the problem of vanishing detail exists in high precision agricultural models. The Small Object Detection (SOD) problem is a cornerstone of precision agriculture, where satellite and Unmanned Aerial Vehicle (UAV) imagery must be analyzed to identify individual crops, pests, or irrigation leaks. In these scenarios, the objects of interest often occupy less than 0.1% of the total image area [2].

To solve this, agricultural models employ a Tiling (or Patch-based) Inference strategy [2], [3]. Instead of global resizing, the image is partitioned into a grid of overlapping tiles. This preserves the Ground Sampling Distance (GSD); the physical distance on the ground represented by a single pixel ensuring that the spatial features required for detection remain intact. Research in this field, such as the Slicing Aided Hyper Inference (SAHI) framework [3], demonstrates that tiling can increase the average precision of small object detection by over 20-30% compared to standard resized inference.

But these solutions are often specialized and computationally complex, typically designed for offline processing on powerful workstations rather than real-time edge deployment.

Growing AI and Evolving Edge Computing

The rapid proliferation of Artificial Intelligence (AI) has led to a paradigm shift in how data is processed, moving from centralized cloud architectures to the Edge. As of 2025, it is estimated that over 75% of enterprise-generated data is created and processed outside traditional data centers [4]. This evolution is driven by the necessity for real-time responsiveness in applications such as autonomous surveillance and industrial monitoring, where the latency of round-trip cloud communication is no longer acceptable [5].

Modern edge computing involves a decentralized network of edge nodes ranging from Internet of Things (IoT) gateways to specialized hardware accelerators like NVIDIA Jetson modules that perform inference locally. However, the increasing complexity of state-of-the-art CNNs often exceeds the memory and thermal envelopes of a single edge device. This has birthed the concept of Distributed Edge Inference, where a singular heavy workload is partitioned across multiple heterogeneous nodes [6].

1.2. Motivation

The motivation for this research stems from the widening gap between the increasing resolution of modern image sensors and the static input constraints of high-performance deep learning models. While 4K and 8K cameras are becoming standard in security

and industrial monitoring, the hardware tax required to process these images at native resolution is prohibitive for most edge-deployed systems. Current solutions generally follow one of two suboptimal paths: aggressive downsampling, which sacrifices the high frequency spatial data necessary to detect small or distant subjects, or expensive hardware scaling, which relies on power hungry Graphics Processing Units (GPUs) that are unsuitable for remote or battery operated environments. There is a critical need for a middle ground—an architecture that preserves the raw detail of the sensor while distributing the computational burden across a modular network.

1.3. Problem Definition

The core challenge addressed in this research is the Information-Computation Paradox in high-resolution computer vision. As sensor technology advances, the gap between captured data density and real-time processing capability continues to widen. This problem is formally defined through three primary constraints:

The Scaling Bottleneck and Receptive Field Limitations: Most state-of-the-art CNN architectures, such as ResNet or YOLO variants, utilize fixed input dimensions (e.g., 416×416 or 640×640). When a 4K UHD stream is force-fitted into these dimensions, the resulting bilinear or bicubic interpolation acts as a low-pass filter, effectively erasing the high-frequency components that define small-scale objects. Consequently, the network's effective receptive field becomes larger than the object itself, leading to a failure in feature activation [1].

Resource Heterogeneity and Memory Constraints: Edge devices often lack the unified memory architecture required to load large-scale models alongside high-resolution frame buffers. A monolithic inference approach on a single node leads to Out of Memory (OOM) errors or thermal throttling. There is currently a lack of standardized frameworks that allow a single inference task to be decomposed spatially across a cluster of heterogeneous low-power devices [7].

The Aggregation Challenge: Splitting an image into tiles introduces boundary artifacts. If a human subject is positioned across the intersection of four tiles, no single node possesses the complete spatial context to identify the subject. The problem, therefore, is not only how to distribute the tiles but how to design an Aggregator Node capable of fusing partial feature maps (logits or latent vectors) into a coherent global detection without re-introducing the original computational bottleneck [8].

1.4. Objectives

- To develop a light-weight model architecture to process a high resolution image by dividing into low-resolution tiles.
- To detect human presence by implementing the developed model using distributed architecture of resource constrained edge devices.

1.5. Scope and Applications

The scope of this project is centered on the intersection of distributed systems and embedded computer vision. While the methodology is developed using HPD as the primary use case, the underlying architecture for spatial partitioning and feature aggregation is designed to be model-agnostic.

By enabling high-resolution sensing on low-cost, low-power hardware, this project opens new avenues for several critical domains:

- **High-Precision Industrial Safety:** Monitoring large-scale factory floors where small-scale anomalies (e.g., a hand near a restricted gear) must be detected at high resolution without the cost of high-end industrial Personal Computerss (PCss).
- **Privacy-Preserving Smart Homes:** Since tiles are processed locally and only feature maps (not raw images) are aggregated, the system inherently provides a layer of data privacy, making it ideal for residential presence sensing and elderly care monitoring.
- **Energy Management in Heating, Ventilation, and Air Conditioning (HVAC) Systems:** In large-scale commercial buildings or smart offices, precise occupancy detection is critical for optimizing HVAC loads. Traditional Passive Infrared (PIR) sensors often fail to detect stationary occupants, leading to energy inefficiencies. A distributed, tile-based CNN can monitor vast open-plan spaces with high fidelity, detecting occupants even in seated or obscured positions. By feeding real-time presence data into a Building Management System (BMS), the HVAC system can dynamically adjust airflow and temperature in specific zones, significantly reducing the building's carbon footprint and operational costs.

The boundaries of this study are limited to static or slow-moving subjects where the latency of distributed aggregation does not exceed the temporal changes in the scene.

2. LITERATURE REVIEW

The shift toward high-fidelity occupancy sensing is driven by the failure of traditional hardware to detect stationary presence. Ahmad et al. [9] discusses the integration of hybrid PIR and Radar systems to improve detection reliability in smart buildings. The study finds that while hybrid sensors reduce false triggers, they still exhibit nearly 0.0% reliability when occupants remain perfectly still. However, the paper lacks a solution for low-power visual confirmation, leaving a gap that only computer vision can fill by analyzing physiological micro-movements.

To deploy vision models on edge hardware, extreme model compression is required. Nagel et al. [10] provides a comprehensive white paper on neural network quantization, specifically focusing on 8-bit integer (INT8) Post-Training Quantization. Their research finds that converting models to INT8 reduces memory footprints four-fold with less than a 1% drop in accuracy. Despite these gains, the paper lacks specific implementation strategies for the 512 KB SRAM limits of the ESP32-S3 [11], which requires specific instruction-set optimizations.

Further optimization of model response is explored by Wu et al. [12] through the Q-CNN framework. The paper talks about prioritizing response fidelity matching the output of quantized layers to the original floating-point layers rather than just minimizing weight error. They find that this approach allows for up to 20x model compression while maintaining high classification performance. However, the study lacks a distributed deployment strategy, meaning the model still struggles to process high-resolution images on a single low-power node.

Architectural efficiency is a core theme in reducing parameter counts. Iandola et al. [13] introduces the SqueezeNet architecture, which utilizes Fire Modules to replace expensive filters with 1×1 squeeze layers. The study finds that SqueezeNet achieves AlexNet-level accuracy with 50x fewer parameters, making it ideal for mobile deployment. While efficient, the architecture lacks the ability to handle extremely low-bit operations, which are necessary for the ultra-low-power requirements of this specific project.

To push efficiency further, Juefei-Xu et al. [14] proposes Local Binary Convolutional Neural Networks (LBCNN). The paper talks about replacing standard learnable filters with fixed, sparse binary kernels that do not require weight updates during training. They find that this method achieves a massive 169x parameter reduction, significantly lowering inference latency. The primary drawback is that LBCNN lacks a native method for processing ultra-high-resolution (4K) frames, which typically causes memory overflows on microcontrollers.

Processing high-resolution data requires spatial decomposition. Li et al. [15] discusses data partitioning strategies to handle large tensors in deep learning workflows. Their findings suggest that splitting images into a grid (tiling) preserves high-frequency spatial details that are usually lost when an image is resized to fit a standard model. However, the paper focuses on high-end GPU clusters and lacks a lightweight communication protocol designed for resource-constrained micro-clusters.

Small object detection is specifically addressed by Akyon et al. [16] through the SAHI (Slicing Aided Hyper Inference) framework. The paper talks about slicing high-resolution images into overlapping patches to help the model see smaller details at a distance. They find that SAHI increases detection accuracy by 20–30% for small objects. The main limitation is the high computational overhead, as the framework is designed for powerful workstations rather than real-time edge devices.

Finally, the communication between edge nodes is critical for a distributed system. Jeong et al. [17] evaluates low-latency communication protocols for IoT device clusters. Their research finds that peer-to-peer protocols can achieve latencies as low as 1.115 ms, which is sufficient for real-time synchronization. However, the study lacks an application-specific test for image tile transmission, which involves higher data throughput than simple sensor packets. This project fills these combined gaps by deploying a real-time, distributed tiling system on an ESP32 cluster.

3. REQUIREMENTS ANALYSIS

The development of a distributed, tile-based HPD system requires a multi-faceted analysis of hardware constraints, software dependencies, and network reliability. The section below discusses the basic requirements needed for the project to be feasible.

3.1. Hardware Requirements

Given the ESP32-S3 chip in the design, the choice of hardware is based on the speed-up and power consumption needs realted to AI.

- **Camera Module:** A high-resolution camera module(OV2640) capable of capturing images at higher resolution than traditional image processing systems, which will then be partitioned.
- **Input Node:** A master node responsible for taking images from the image sensor; partitioning them into multiple tiles and distributing them to edge nodes.
- **Edge Computing Nodes:** Multiple ESP32-S3 MCUs are required. These must support the $XtensaLX7$ vector instructions to handle the tile-level CNN inference.
- **Aggregator Node:** A simple ESP32 responsible for collecting feature vectors and executing the final classification layer.
- **Communication Interface:** A low-latency communication system for inter-node communication Espressif Non-Beaconing Wirelesss (ESP-NOWs).

3.2. Software & Framework Requirements

To achieve ultra-low bit quantization and distributed processing, the following software stack is required:

- **Development Environment:** Arduino IDE using C/C++.
- **AI Inference Library:** The Espressif Deep Learning library (ESP-DL) makes use of effiecient kernels for quantization 8-bit or 16-bit computation.
- **Model Optimization:** Initial training and conversion of models with TensorFlow Lite for Microcontrollers (TFML) are followed by Quantization-Aware Training (QAT).
- **Tiling Logic:** A custom C++ pre-processing module to handle spatial partitioning and memory-safe buffer management for individual tiles.

3.3. Non-functional Requirements

- **Latency:** There will be low latency in order for this to be a real-time application.
- **Scalability:** This would require an adjustable number of nodes, depending on the level of resolution and required Frames Per Second (FPS).

- **Robustness:** If one of the nodes is no longer working, it will need to fall back to the other nodes to get partial occupancy information.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. Model Selection and Optimization Pipeline

4.1.1. MobileNetV2 Architecture

The proposed system leverages a CNN based on a mobilenetv2 backbone, specifically optimized for binary classification tasks where $f_\theta : \mathbb{R}^{N\times1\times H\times W} \rightarrow \mathbb{R}^{N\times2}$. The architecture is designed to balance high-level feature extraction with the strict hardware constraints of MCUs.

Grayscale Adaptation and Transfer Learning

To minimize memory consumption and computational latency, the model is adapted to process single-channel grayscale images. Since the pretrained weights were originally optimized for rgb data, the first convolution layer's weights ($W \in \mathbb{R}^{C_{out}\times3\times k\times k}$) are averaged across the channel dimension:

$$Y = 0.299R + 0.587G + 0.114B \tag{4-1}$$

This technique retains low-level feature detection, such as edge detectors and textures, while allowing the model to function on simplified Static Random-Access Memory (SRAM)-efficient inputs.

Classifier Head and Loss Function

The final classification is performed by a narrow bottleneck head ($1280 \rightarrow 64 \rightarrow 2$) using ReLU activation and dropout (0.2 and 0.1) to prevent overfitting. The resulting logits $\hat{y}$ are evaluated using Cross-Entropy Loss:

$$L = -\sum_{i=1}^{2} y_i \log\left(\frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}\right) \tag{4-2}$$

4.1.2. Local Binary Convolution Based Architecture

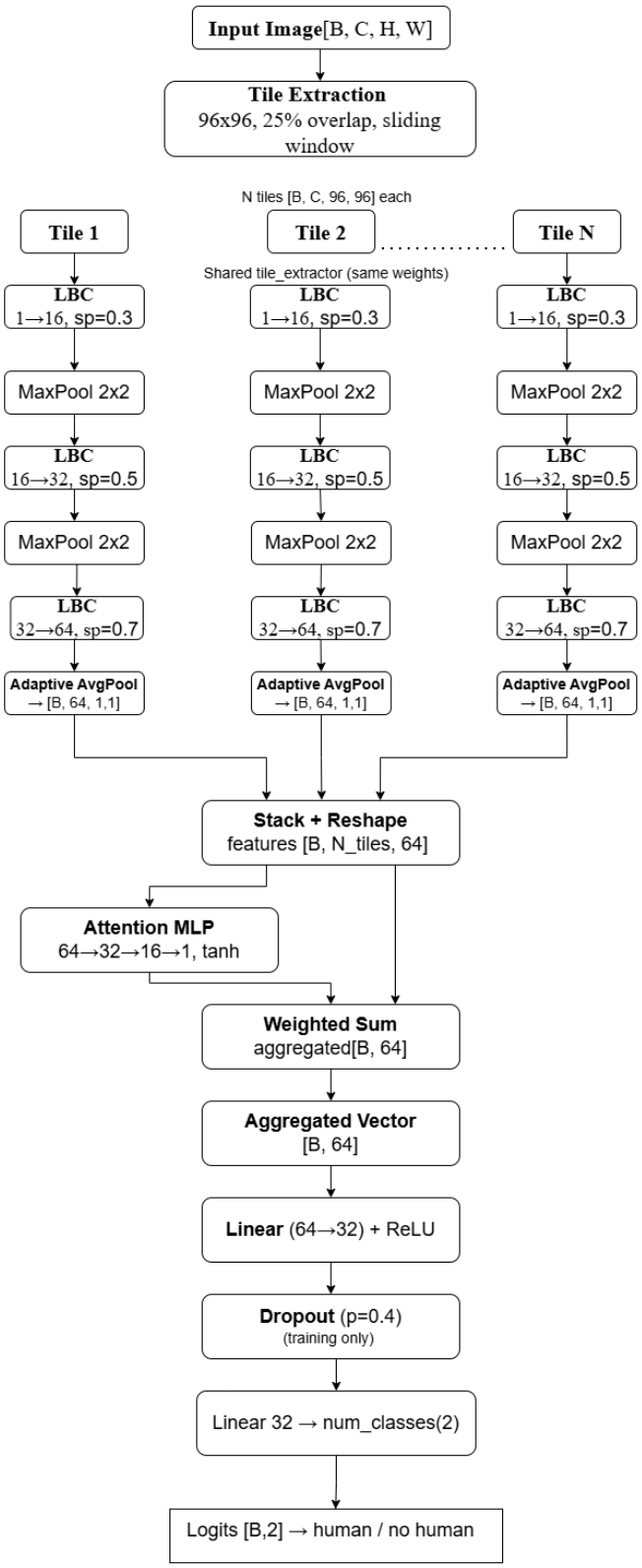


Figure 4-1. Multi-Tile Attention-Based CNN Architecture

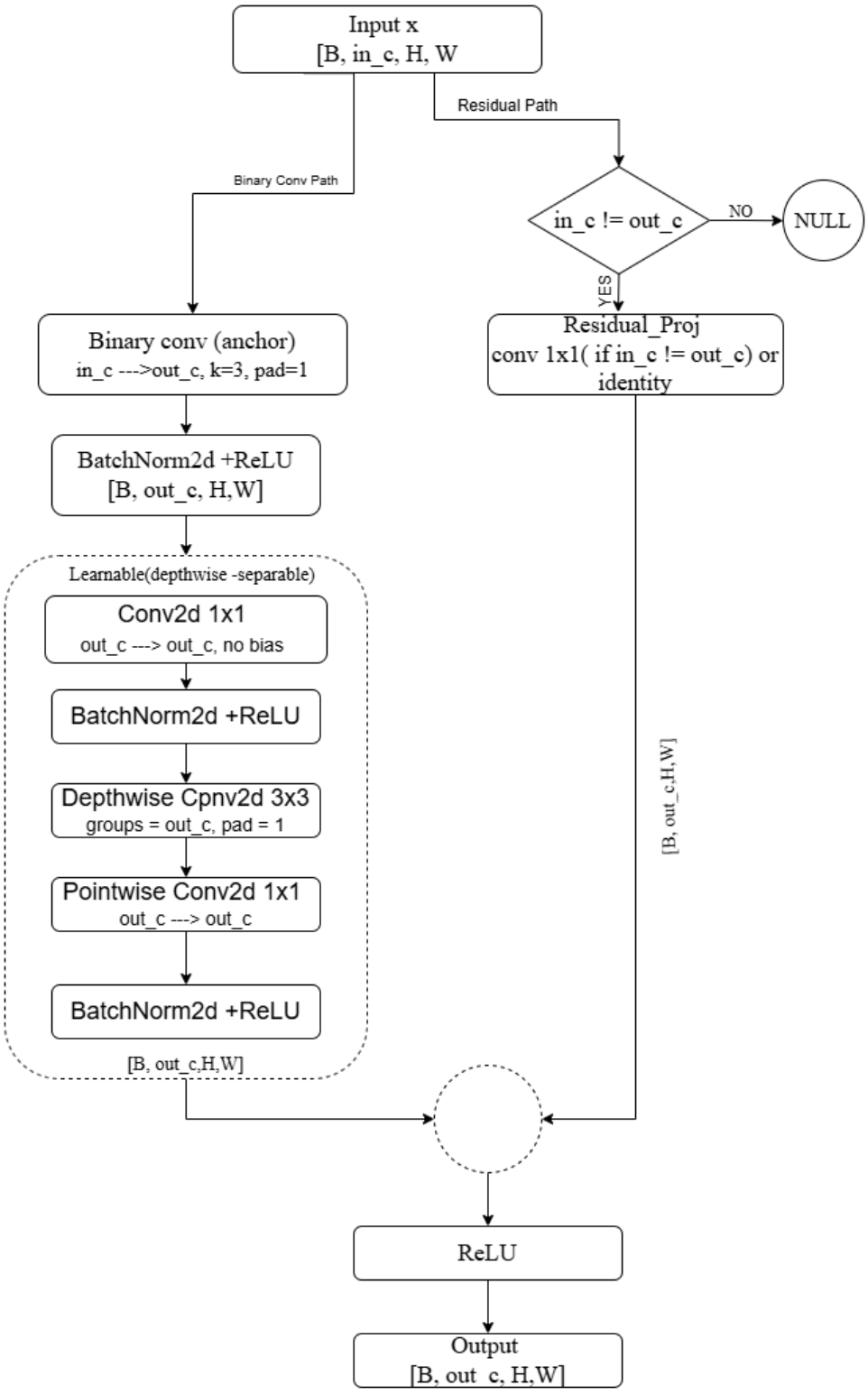


Figure 4-2. Detailed Schematic of the Local Binary Convolution Module

LBCLayer Overview

The LBCLayer is a hybrid convolutional layer designed to combine fixed binary convolutions (anchors) for efficient sparse structural feature capture with a learnable 1×1 convolution for task-specific adaptation. It is particularly suitable for resource-efficient networks and sparse feature representations, inspired by research on Local Binary Convolutions (LBC) and sparse binary kernels.

Mathematically, for an input $x \in \mathbb{R}^{N \times C_{in} \times H \times W}$, the output is defined as:

$$x_{\text{out}} = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x))) \tag{4-3}$$

Where:

- $\text{Conv}_{\text{binary}}$ uses fixed ternary weights $\in \{-1, 0, 1\}$.
- Conv1x1 is trainable and combines channels efficiently.

LBCLayer Components

1. Fixed Anchor Convolution

The purpose of this component is to extract general features without learning, thereby reducing the parameter count.

- **Weight generation**: The first step is the creation of weights matrix $W \sim \mathcal{N}(0, 1)$ and its sparsifying with parameter s:

$$\text{binary\_weight}_{i,j,k,l} = \begin{cases} 1 & \text{if } W_{i,j,k,l} > \text{threshold} \\ -1 & \text{if } W_{i,j,k,l} < -\text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

- The sparsity parameter s will define how many zeros will there be in the matrix.
- This matrix of weights is saved as a buffer (`register_buffer`), which means it won't be trained afterwards.

The main idea is to capture general directional/edge patterns efficiently similar to handcrafted filters like Sobel, but stochastically generated.

2. Non-linearity (ReLU)

After the fixed binary convolution, non-linearity is applied by ReLU: $x' = \text{ReLU}(x)$. Non-linearity is introduced to help approximate complex functions, which also maintain sparsity and improving computational efficiency since there will be a lot of zeros.

3. Learnable 1x1 Convolution

The 1×1 kernel combines sparse feature maps into task-relevant channels while keeping computational costs low.

$$x_{\text{final}} = W_{1\times1} * x' \tag{4-4}$$

Only this part is trainable, allowing the layer to adapt to specific tasks like human detection.

Forward Pass Flow

For a single LBCLayer, the flow is as follows:

1. The input to the layer is $x \in \mathbb{R}^{N\times C_{in}\times H\times W}$.
2. Perform binary convolution using fixed parameters: $x_1 = \text{Conv}_{\text{binary}}(x)$.
3. Use ReLU activation function: $x_2 = \text{ReLU}(x_1)$.
4. Perform 1x1 convolution using learnable parameters: $x_{\text{out}} = \text{Conv1x1}(x_2)$.
5. Output: $x_{\text{out}} \in \mathbb{R}^{N\times C_{\text{out}}\times H\times W}$.

Advantages of LBCLayer

- **Computational efficiency**: Fixed sparse weights reduce multiply-adds, and 1×1 learnable convolutions are computationally cheap.
- **Stability**: Binary anchors are not updated, which avoids noisy gradients; the network only optimizes the final 1×1 convolution.
- **Sparsity Control**: The sparsity parameter allows control over network complexity and feature density, encouraging sparse representations that improve generalization.
- **Compatibility with Tilings**: LBCLayer is very well suited for tiling based systems such as HumanDetector where individual tiles can be processed separately before being assembled.
- **Biological Inspiration**: Sparse and binary inspired filters mimic biological visual cortex processing that is helpful for edge detection and texture recognition.

Mathematical Summary

Considering the inputs x, the summary of the operation and gradients are as follows:

$$\begin{aligned} \text{Anchor weight: } A &= f_{\text{binary}}(W, s) \\ x_1 &= x * A \\ x_2 &= \text{ReLU}(x_1) \\ x_{\text{out}} &= x_2 * W_{1\times1} \end{aligned}$$

Gradients: $\frac{\partial L}{\partial W_{1\times1}} \neq 0$, whereas $\frac{\partial L}{\partial A} = 0$. In this case, $*$ denotes a convolution, s denotes the sparsity threshold, and A denotes the fixed binary anchor weights.

Overview of HPD

The HPD (HumanDetector) is a **tile-based convolutional network** designed to detect humans in images. Its key characteristics:

1. **Inputs:** $x \in \mathbb{R}^{N\times1\times H\times W}$ grayscale images.
2. **Tiling:** Divide image into tile_rows × tile_cols patches without overlaps.
3. **Per-tile Feature Extraction:** The use of LBCLayer to perform sparse and task-aware feature extraction from each patch.
4. **Features Aggregation / Stitching:** Features from all the patches aggregated into one feature vector.
5. **Classifier Head:** The output dense layers that provide logit values for `num_classes`. (E.g., Human vs. Not Human).

Mathematically:

$$f_\theta(x) = \text{Classifier}\left(\text{Concat}_{i,j}(\text{TileExtractor}(x_{i,j}))\right) \tag{4-5}$$

Where $x_{i,j}$ is the (i, j)-th tile.

Tiling Stage

The tiling stage prepares the input for localized processing.

- **Input:** $x \in \mathbb{R}^{N\times1\times H\times W}$
- **Parameters:** tile_rows, tile_cols
- **Tile size:**

$$h = \frac{H}{\text{tile\_rows}}, \quad w = \frac{W}{\text{tile\_cols}} \tag{4-6}$$

- **Split input into tiles:**

$$x_{i,j} = x[:, :, i \cdot h : (i+1) \cdot h, j \cdot w : (j+1) \cdot w] \tag{4-7}$$

Purpose:

- Localizes feature extraction → each tile acts as a **node** (MCU-like processing).
- Improves **spatial granularity** for detecting humans.
- Tiles are **independent**, can be processed in parallel.

Tile Feature Extractor (LBC-based)

Every tile goes through *tile_extractor* whose structure is described below:

- **LBC Layer 1:** $1to16$ channels with sparsity = 0.9.
- **MaxPool2d:** Kernel size 2 by reducing spatial dimensions by half.
- **LBC Layer 2:** $16to32$ channels with sparsity = 0.9.
- **AdaptiveAvgPool2d:** pooling spatial dimensions to 1×1.

For individual tile $x_{i,j}$:

$$x' = \text{LBCLayer}(x_{i,j}) = \text{Conv1x1}(\text{ReLU}(\text{Conv}_{\text{binary}}(x_{i,j}))) \tag{4-8}$$

Aggregation / Stitching

After feature extraction, the tile outputs get aggregated together:

1. **Flatten the output for each tile:** $f_{i,j} \in \mathbb{R}^{32}$
2. **Concatenate all tiles:** If number of tiles = tile_rows × tile_cols:

$$f_{\text{combined}} = [f_{1,1}, f_{1,2}, \ldots, f_{\text{tile\_rows,tile\_cols}}] \in \mathbb{R}^{32 \cdot \text{num\_tiles}} \tag{4-9}$$

This process can be considered analogous to graph aggregation, treating every tile as a graph node

Classifier Head

The classifier performs an MLP on the concatenated vector:

- **Input:** $f_{\text{combined}} \in \mathbb{R}^{32 \cdot \text{number of tiles}}$
- **Layers:** Linear($32 \cdot$ number of tiles $\rightarrow 64$) $\rightarrow$ ReLU $\rightarrow$ Linear($64 \rightarrow$ num_classes)

- **Output Classificaation Logits:**

$$\hat{y} \in \mathbb{R}^{N \times \text{num\_classes}} \tag{4-10}$$

Uses **CrossEntropyLoss** when training:

$$L = -\sum_{c=1}^{\text{num\_classes}} y_c \log \text{softmax}(\hat{y}_c) \tag{4-11}$$

Computational Efficiency

The LBCLayer reduces Floating Point Operationss (FLOPss) significantly:

$$\text{FLOPs} \approx C_{in} \cdot C_{out} \cdot k^2 \cdot H \cdot W \tag{4-12}$$

- **Binary conv:** Multiplications are basically simple additions/subtractions.
- **1x1 conv:** Provides lightweight mixing of channels.
- **Tile-based processing:** Can be parallelized across multiple nodes.

Mathematical Summary of Forward Pass

1. **Tile splitting:**

$$\{x_{i,j}\}_{i,j=1}^{R,C} = \text{Tile}(x) \tag{4-13}$$

2. **Feature extraction (per tile):**

$$f_{i,j} = \text{AdaptiveAvgPool2d}(\text{LBCLayer2}(\text{MaxPool2d}(\text{LBCLayer1}(x_{i,j})))) \tag{4-14}$$

3. **Flatten and concatenate:**

$$f_{\text{combined}} = \text{Concat}([f_{i,j}]) \tag{4-15}$$

4. **Classification:**

$$\hat{y} = \text{Linear}_2(\text{ReLU}(\text{Linear}_1(f_{\text{combined}}))) \tag{4-16}$$

5. **Loss:**

$$L = -\sum_{c=1}^{\text{num\_classes}} y_c \log \text{softmax}(\hat{y}_c) \tag{4-17}$$

Advantages of HPD Architecture

1. **Sparse and efficient:** LBC reduces compute and memory footprint.
2. **Tile-based learning:** Learns local patterns for small human parts.

3. **Flexible:** Can adapt tile size and sparsity based on resources.
4. **Parallelization:** Processed independently per tile.
5. **Low-data scenario:** Set anchors minimize training needs.

4.2. Hardware Requirements and system configuration

The design employs a distributed architecture that is multi-node based on the ESP32 and ESP32-S3 MCU. In order to maximize efficiency and take advantage of limited resources, the system is structured hierarchically in three levels.

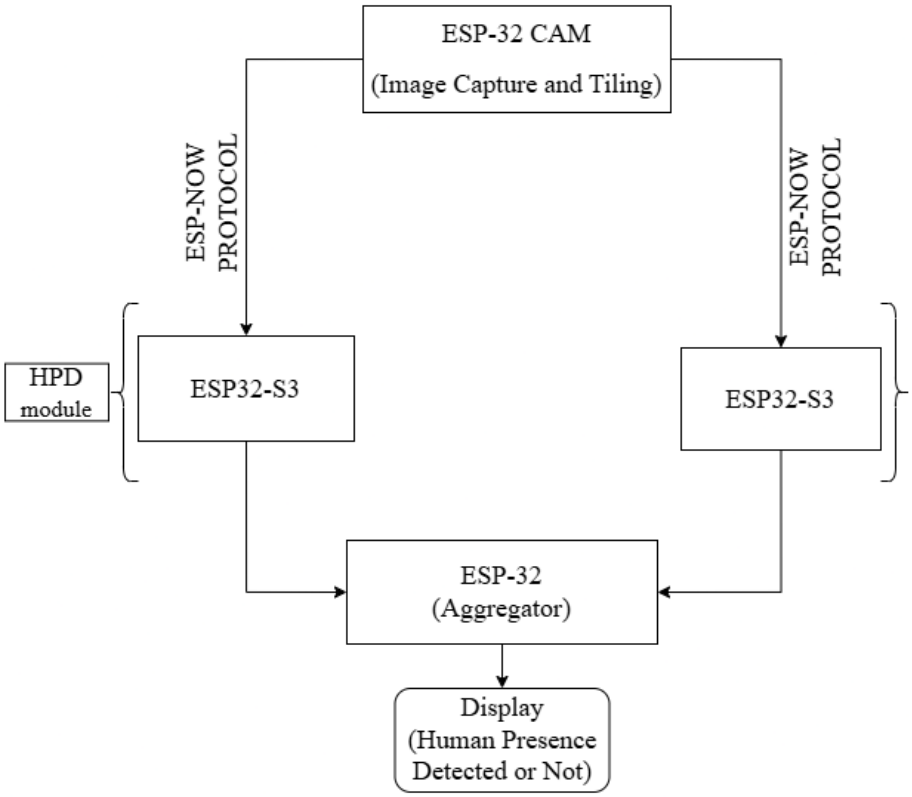


Figure 4-3. System Architecture showing main components and their interactions.

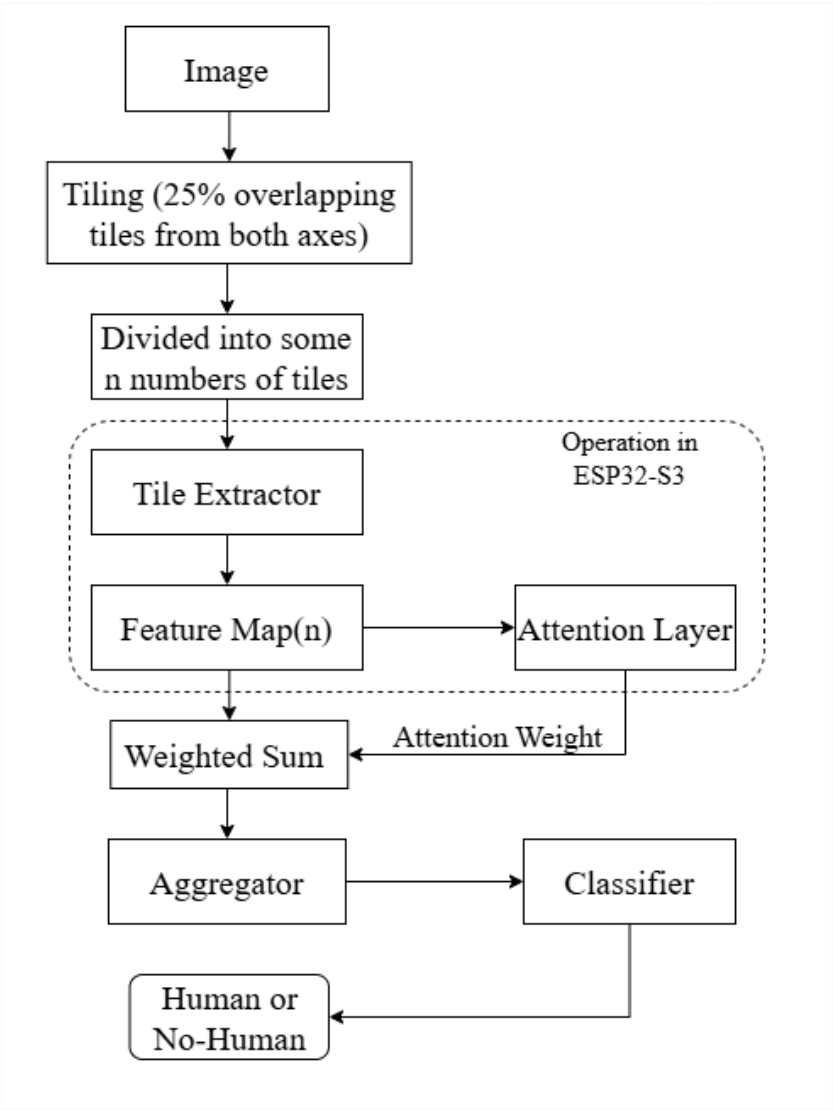


Figure 4-4. Detailed Data Flow Diagram of the Tiling and Inference Pipeline.

4.2.1. Input Layer

Centered around an ESP32-CAM [18], this layer manages high-bandwidth data acquisition and initial pre-processing. Its primary role is to prepare and distribute workloads to the processing tier through the following pipeline:

Luminance Conversion (Grayscale)

Processing three-channel RGB data significantly inflates the memory buffer requirements and computational complexity ($O(3n)$ vs $O(n)$). The captured frame has to be converted to an 8-bit grayscale format using a weighted luminance formula as given in (4-18) [19].

$$P(y = 1 \mid x) = \frac{e^{z_{\text{person}}}}{e^{z_{\text{bg}}} + e^{z_{\text{person}}}} \tag{4-18}$$

Where z_{person} and z_{bg} represents the logits corresponding to "person" and "background" class, respectively. Data compression reduces the amount of data sent by two-thirds, resulting in a massive reduction of memory space required in the processing nodes. Such data compression technique reduces the input data volume by 66.6%, hence leaving more room for heap memory allocation for tiling and distribution buffers.

$$Y = 0.299R + 0.587G + 0.114B \tag{4-19}$$

Partitioning Distribution

The process begins with the Tiling module, which extracts n overlapping tiles from the source image. This step is critical for maintaining high-resolution features on resource-constrained hardware. By using a 25% overlap between adjacent tiles, the system ensures that human subjects positioned at tile boundaries are not bisected, preserving spatial continuity for the feature extractor.

4.2.2. Communication Protocol Implementation

The Espressif Systems (ESP) protocol will be the communication backbone [20], selected for its highly efficient, peer-to-peer data exchange capabilities and low-latency feedback. While ESP is efficient, its throughput is limited (observed maximums up to 771 Kilobyte (KB)/s). Therefore, the implementation will rely on efficient data packing, ensuring that tile data transfer maximizes the available throughput and that result packets adhere to the small packet size limits (< 250 bytes for control and aggregation data). The ability to transfer data quickly is paramount, as the latency gained by parallel processing can be quickly lost if the communication time dominates the total system latency.

4.2.3. Feature Extraction Layer

- Parallel Processing: Each node receives a specific low-resolution tile from the Input Layer.
- Local Inference: Each node runs the HPD model on its assigned tile, extracting features and computing an attention weight that indicates the likelihood of human presence in that tile.
- Output Generation: Each node outputs a feature map and an attention weight, which are then transmitted to the Output Layer for aggregation and final decision-making.

4.2.4. Output Layer (Aggregation & Logic)

As the final stage of the architecture, this layer serves as the central decision-making layer. It receives the feature map and attention weight for each tile from feature extraction nodes and performs the following operations:

- **Attention Score Calculation:** The attention score for each tile is computed using the SoftMax function, which normalizes the quantized scores into a probability distribution. This allows the system to weigh the importance of each tile's features when making the final decision.
- **Weighted Feature Aggregation:** The attention scores are used to perform a weighted sum of the feature maps from all tiles, effectively combining the information while giving more weight to tiles that are more likely to contain human features.
- **Final Classification:** The aggregated feature map is passed through a classifier layer (e.g., a fully connected layer) to compute a confidence score for human presence in the entire scene. This score is then used to make a binary decision (human detected or not) and can be displayed on an output device such as an LCD screen.

4.3. Hardware Used

The core hardware of this project is built around ESP32-family MCUs. It is chosen because they are low-cost, low-power, and capable of running edge-level machine learning tasks. This system follows a distributed architecture. Here, image capturing, processing and decision-making are separated across different ESP32 devices to overcome memory and processing limitations of a single microcontroller. This hardware design allows the system to perform real-time HPD without relying on cloud servers, making it faster, more private, and more energy efficient.

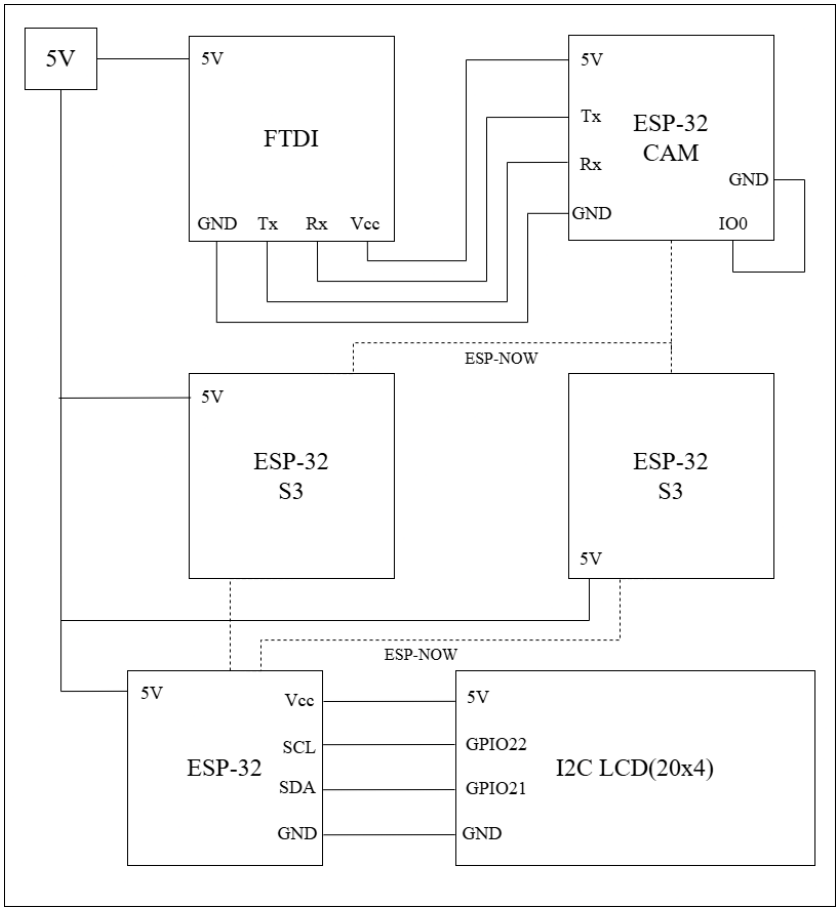


Figure 4-5. Wiring Interface between Programming (FTDI), Processing (ESP32-S3), and Visualization (I2C LCD) Units.

The hardware architecture as shown in 4-5 consists of a distributed processing network designed for real-time data acquisition and visualization. The system is partitioned into three primary functional blocks: the Sensing Node (ESP32-CAM), the Processing Cluster (ESP32-S3 units), and the Output Interface (I2C LCD connected via ESP32)

4.3.1. ESP32-CAM

The ESP-CAM module is used as the input hardware of the system. Its main role is to capture visual data from the environment. Here, the camera continuously captures images of 224×224 at a controlled rate of 1 every 15 seconds. Since color images require more memory and computation, the ESP-CAM converts the captured rgb images into grayscale format. After this, the high-resolution image is divided into smaller tiles of 96×96 with overlap of 25% between adjacent tiles. These smaller image tiles are easier to process and then transmit to other ESP devices using ESP-NOW protocol for further processing. Here, the tiles are evenly distributed between the two esp32s3 devices when the total number of tiles is even; if the number of tiles is odd, the first node is assigned one additional tile.

4.3.2. USB to TTL converter (FTDI)

The USB to TTL (FTDI) converter is used to connect the ESP32-CAM boards to a laptop for programming. It is because ESP32-CAM does not have a built-in USB interface for programming or serial communication. The FTDI module acts as a bridge between them since a laptop communicates using USB and the ESP32 uses TTL serial (UART) signals. In this project, the FTDI also supplies 5V power from the laptop via USB, which removes the need for an external power source.

Connect GPIO0 to GND in ESP32-CAM for flashing. This indicates that the ESP32-CAM is ready to receive new code. After uploading the code, we can disconnect GPIO0 from GND.

Table 4-1. Wiring the FTDI to ESP32-CAM

FTDI	**ESP32-CAM**
VCC	5V
GND	GND
RX	U0T
TX	U0R

4.3.3. ESP32-S3

The ESP32-S3 MCUs act as the processing or inference nodes in the system. Each ESP32-S3 receives a small image tile from the ESP32-CAM and runs a Tiny Machine Learning (TinyML)-based HPD model on it. These boards are specially selected because they are optimized for AI tasks and support low-bit operations. These are essential for running neural networks within limited memory. Here, each ESP32-S3 processes its assigned tile independently and determines whether human features are present. Instead

of sending large image data back, it sends only a compact result, such as a confidence score and tile location, which minimizes communication overhead.

4.3.4. ESP32

The ESP32 board is used as the aggregation node of the system. It is used to collect the results from all ESP32-S3 and uses attention weight from all tiles to compute attention score using SoftMax function and then uses the attention score to perform weighted sum of feature maps of tiles. Then, it passes the new feature map through classifier layer to compute the confidence score. This confidence score is to form a final decision about human presence in the entire scene.

4.3.5. I2C LCD Display

In this system, the I2C LCD display is used to show the final output of the project. After the ESP32-S3 boards finish processing the image tiles and send their results, the ESP32 combines all the information and decides whether a human is present or not. This final decision is then shown on the LCD screen as **"Human Detected"** or **"No Human Detected".** The I2C LCD has been chosen because it needs only two wires for communication. It keeps the wiring simple and saves GPIO pins on the ESP32.

Table 4-2. Wiring the I2C LCD to ESP32

I2C LCD Pin	ESP32 Pin	Purpose
VCC	3.3V or 5V	Power Supply
GND	GND	Ground Connection
SDA	GPIO21	Data Line
SCL	GPIO22	Clock Line

4.3.6. Power Supply

All the hardware components in the system are powered using a laptop via USB connections. The laptop provides a stable 5V power supply to the ESP32-CAM, ESP32-S3, and ESP32 boards during development and testing. This simplifies the setup, eliminates the need for external power adapters, and allows easy monitoring and debugging through serial communication.

4.3.7. Low-Bit Quantization Implementation

The baseline operational model will utilize Post-Training Quantization (PTQ) to convert the HPD model to symmetric 8-bit Integer (INT8) weights and asymmetric INT8 activations. This approach is selected for its demonstrated success in significantly reducing computational complexity while maintaining high classification reliability in resource-constrained deployments. Further optimization will involve the experimental

deployment of 4-bit Integer (INT4) quantization for specific non-critical layers. This mixed-precision approach will push memory reduction to its architectural limits, targeting a final Flash footprint of < 1.0 Megabyte (MB).

To mitigate the precision loss associated with such aggressive compression, calibration datasets will be utilized in conjunction with layer-wise equalization techniques. These methods balance weight ranges across the custom architecture, aiming for maximum accuracy retention (targeting lesser than 90% of the pruned 32-bit Floating Point (FP32) baseline). This optimization strategy is essential to ensure that the model remains robust for presence detection while fitting within the 512 KB SRAM limit.

4.4. Evaluation Metrics and Performance Framework

4.4.1. Detection Reliability Matrix (The Deep Learning Layer)

The main goal is to fix the stationary detection issue that PIR sensors have. The following standard computer vision criteria will be used to assess our CNN model:

Recall (Sensitivity)

Measured especially for participants who are motionless to demonstrate the system's capacity to sustain presence signals in situations where movement is minimal. The Equation is as shown in equation (3-3):

$$Recall = TruePositive(TP)/(TP + FalseNegative(FN)) \tag{4-20}$$

Precision

To ensure environmental noise (shadows, HVAC airflow) does not trigger false positives in the model's inference. The Equation is as shown in equation (3-4):

$$Precision = TP/(TP + FalsePositive(FP)) \tag{4-21}$$

4.4.2. Distributed System Efficiency Matrix (The Computing Layer)

We must measure the benefits of distributing the inference burden among several ESP32-S3 nodes using the Distributed Adaptive Tiling (DAT) framework as this project makes use of a Distributed Cluster.

End-to-End Latency (total): The duration between the Master node's picture acquisition and the ultimate detection outcome.

Memory Footprint: Maximum SRAM used by any single node during inference.

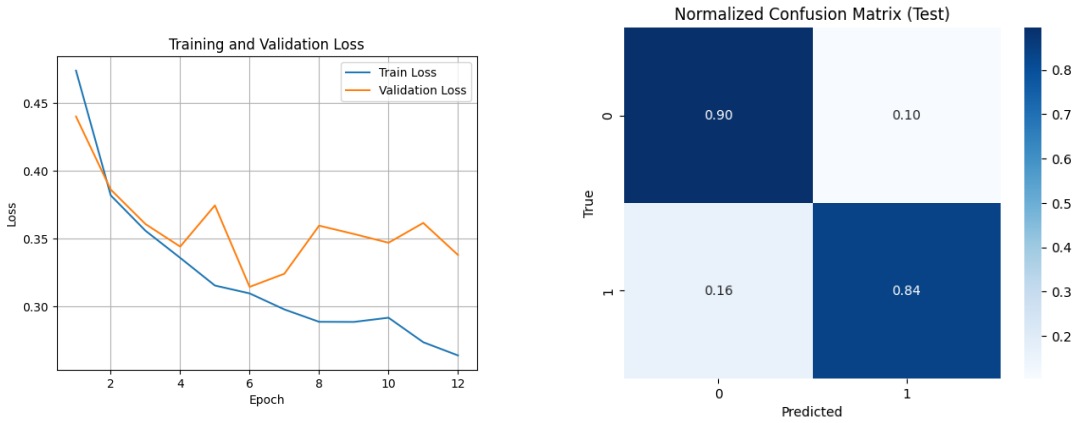
Communication Delay: The proportion of the entire cycle used for local Deep learning computation as opposed to ESP tile transmission.

5. RESULTS AND ANALYSIS

The following report details the training and evaluation of a LBC architecture for HPD. The model was trained over 45 epochs using a *OneCycleLR* scheduler before early stopping was triggered.

5.1. MobileNetV2 Architecture Performance

The utilization of a MobileNetV2 architecture for the task of HPD demonstrated significantly higher accuracy compared to the lightweight Local Binary Convolutional Neural Network (LBCNN) baseline, at the cost of increased model complexity. The model was trained for 12 epochs before early stopping was triggered, using a *OneCycleLR* scheduler with a maximum learning rate of 0.01.



(a) Training and validation loss over 45 epochs.

(b) Normalised confusion matrix on the test set.

Figure 5-1. MobileNetV2 training dynamics and test-set evaluation results.

The model was trained on a significantly larger dataset of 20,736 samples (split into 85% training, 5% validation, and 10% test) compared to the LBCNN baseline. As shown in Figs. 5-1a and 5-1b, both training and validation loss decrease consistently across epochs with strong convergence, indicating that the larger dataset effectively mitigated the overfitting observed in the LBCNN model. The best validation accuracy of **87.34%** was recorded at epoch 7, and early stopping was triggered at epoch 12 with a patience of 5.

Table 5-1. MobileNetV2 Training Run Summary

Metric	Value	Epoch
Best validation accuracy	87.34%	7
Best validation loss	0.3145	6
Final training accuracy	89.32%	12
Test accuracy	86.18%	—
Test Area Under the Curve (AUC) (ROC)	0.931	—
Early stopping	—	12

5.2. MobileNetV2 Test-Set Evaluation

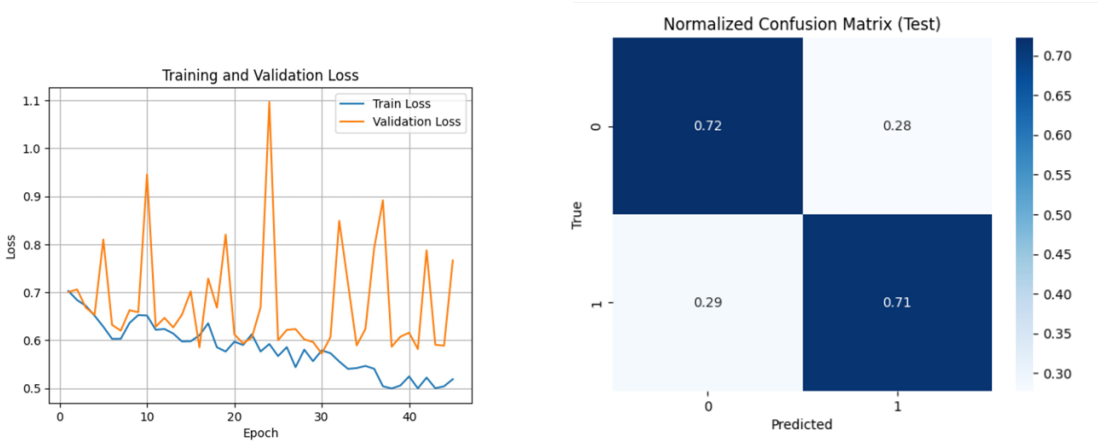
Following training, the best-checkpoint model was evaluated on a held-out test set of 2,432 images. Table 5-2 presents the per-class precision, recall, and F1-score.

Table 5-2. Per-Class Classification Report on the Test Set (n = 2432) for MobileNetV2

Class	Precision	Recall	F1-score	Support
Class 0 (No human)	0.7958	0.8955	0.8427	1005
Class 1 (Human)	0.9193	0.8381	0.8768	1427
Macro average	0.8575	0.8668	0.8598	2432
Weighted average	0.8682	0.8618	0.8627	2432
Overall test accuracy				**86.18%**

5.3. LBCNN Architecture Performance

The utilization of LBC architecture for the task of human presence detection demonstrated the feasibility of a lightweight model with reliable accuracy on resource-constrained MCUs.



(a) Training and validation loss over 45 epochs.

(b) Normalised confusion matrix on the test set.

Figure 5-2. LBCNN training dynamics and test-set evaluation results.

As illustrated in Figs. 5-2a and 5-2b, the LBCNN model shows consistent reduction in training loss over 45 epochs, decreasing steadily from 0.70 to approximately 0.51, while the validation loss remains highly volatile throughout training with sharp spikes reaching as high as 1.10, a behaviour attributed to the limited dataset size of 737 training samples where individual batches exert disproportionate influence on the validation metric. The normalised confusion matrix reveals broadly balanced per-class recall of 0.72 for Class 0 and 0.71 for Class 1, confirming that the weighted sampling strategy successfully avoided majority-class collapse, though the 28% false alarm rate and 29% missed detection rate indicate that the binary convolutional features lack sufficient discriminative precision to cleanly separate the two classes, a direct consequence of the constrained model capacity and limited training data.

Table 5-3. Training Run Summary

Metric	**Value**	**Epoch**
Best validation loss	0.5721	30
Best validation accuracy	77.17%	43
Final training accuracy	78.56%	45
Test accuracy	71.74%	—
Test AUC (roc)	0.777	—
Early stopping	—	45

5.4. Test-Set Evaluation

Following training, the best-checkpoint model was evaluated on a held-out test set of 92 images. Table 5-4 presents the per-class precision, recall, and F1-score.

Table 5-4. Per-Class Classification Report on the Test Set (n = 92)

Class	**Precision**	**Recall**	**F1-score**	**Support**
Class 0 (No human)	0.6190	0.7222	0.6667	36
Class 1 (Human)	0.8000	0.7143	0.7547	56
Macro average	0.7095	0.7183	0.7107	92
Weighted average	0.7292	0.7174	0.7203	92
Overall test accuracy				**71.74%**

5.5. Comparative Analysis: LBCNN vs. MobileNetV2

Table 5-5 summarises the key performance and resource metrics for both architectures evaluated in this work. The LBCNN model was designed for direct deployment on resource-constrained MCUs, while MobileNetV2 serves as a higher-capacity reference baseline trained on a larger dataset.

Table 5-5. Performance Comparison: LBCNN vs. MobileNetV2

Metric	**LBCNN**	**MobileNetV2**
Best validation accuracy	77.17%	87.34%
Best validation loss	0.5721	0.3145
Final training accuracy	78.56%	89.32%
Test accuracy	71.74%	86.18%
Test AUC (ROC)	0.777	0.931
Class 0 Precision	0.619	0.796
Class 0 Recall	0.722	0.896
Class 0 F1-score	0.667	0.843
Class 1 Precision	0.8000	0.9193
Class 1 Recall	0.7143	0.8381
Class 1 F1-score	0.7547	0.8768
Number of parameters	19,779	2,225,858
Model size (FP32)	80 KB	8490 KB
INT8 Model size	20 KB	2122 KB

5.6. Overall Comparative Analysis

The experimental results summarised in Table 5-5 demonstrate a clear performance advantage for MobileNetV2 over the LBCNN across all evaluation metrics. MobileNetV2 achieves a test accuracy of 86.18% and an AUC of 0.931, compared to 71.74% and 0.777 for the LBCNN representing improvements of 14.44 percentage points and 0.154 respectively. This gap is consistent across all per-class metrics, with MobileNetV2 outperforming LBCNN on both precision and recall for both classes.

The difference in training behaviour is equally pronounced. The LBCNN loss and accuracy curves exhibit high volatility throughout all 45 epochs, with validation accuracy oscillating sharply between consecutive epochs. This instability is a direct consequence of the extremely small training set (737 samples), where individual batches exert disproportionate influence on the gradient signal. MobileNetV2, trained on 18,432 samples with ImageNet pretrained weights, converges smoothly within 12 epochs, with both loss and accuracy curves declining and rising monotonically. The pretrained initialisation eliminates the need to learn low-level features from scratch, and the larger dataset provides stable gradients together explaining why MobileNetV2 converges faster, more cleanly, and to a significantly better optimum.

The roc and precision–recall curves reinforce this picture. MobileNetV2's roc curve rises steeply toward the top-left corner, achieving high true positive rates at low false positive rates, reflecting well-separated class probability scores. The LBCNN roc curve rises more gradually, indicating that its 64-dimensional binary feature representations provide weaker class separability. Similarly, MobileNetV2 sustains higher precision across a broader range of recall thresholds, while LBCNN precision drops more sharply as recall increases consistent with a model operating near the limits of its representational capacity.

Despite this performance gap, the comparison must be interpreted in the context of the fundamentally different constraints under which each model operates. MobileNetV2 contains 2,225,858 parameters and occupies 8.49 MB in FP32 far exceeding the memory budget of any commodity MCU. The LBCNN, with only 19,779 parameters and a deployed size of just 0.02 MB after INT8 quantization, fits comfortably within the 512 KB SRAM of the ESP32-S3. The 14.44 percentage point accuracy trade-off is therefore not a shortcoming but a deliberate consequence of designing for deployment on hardware that is approximately **425× more memory-constrained** than what MobileNetV2 requires. Within the scope of real-time, battery-powered, GPU-free edge inference, the LBCNN represents a viable and principled solution to the accuracy–efficiency trade-off inherent in TinyML deployments.

5.7. System Overview and Evaluation Methodology

The distributed inference pipeline was evaluated across four distinct processing stages: frame capture and tiling at the ESP-CAM, tile transmission from the ESP-CAM to the two ESP32-S3 worker nodes, tile-level feature extraction and attention scoring at each worker node, weighted feature transmission from the worker nodes to the ESP32 aggregator, and final classification at the aggregator. Each stage was benchmarked independently using dedicated test sketches with synthetic inputs to isolate stage-specific latency from end-to-end confounding factors. All timing measurements were recorded using the ESP32's hardware *micros()* counter at 240 MHz CPU frequency. Table 5-6 presents the complete latency breakdown across all pipeline stages.

Table 5-6. End-to-End Pipeline Latency Breakdown

Stage	**Node**	**Detail**	**Latency**
Frame capture + tiling	ESP-CAM	224×224 → 9 tiles	4 ms
Tile transmission (CAM → Workers)	Worker 1	5 tiles × 38 fragments	3,243 ms
Tile transmission (CAM → Workers)	Worker 2	4 tiles × 38 fragments	1,183 ms
Tile inference (extractor + attention)	Worker 1	5 tiles sequential	13,284 ms
Tile inference (extractor + attention)	Worker 2	4 tiles sequential	10,627 ms
Feature transmission (Workers → Aggregator)	Per tile avg	69-byte payload	472 ms
Aggregator (softmax + sum + classifier)	ESP32	9 tiles aggregated	0.342 ms
Total pipeline (critical path)	Worker 1 bottleneck		~16,999 ms

5.7.1. ESP-CAM: Frame Capture and Tiling

The ESP-CAM node captures a 224×224 grayscale frame and partitions it into nine overlapping 96×96 tiles using a step size of 72 pixels (25% overlap) in both spatial axes. The complete capture-to-tile pipeline executes in 4 ms, making this stage entirely negligible relative to all downstream processing. This confirms that the tiling strategy itself introduces no meaningful overhead and that the ESP-CAM is capable of sustaining the intended 1 FPS frame rate from a capture perspective alone.

The 4 ms figure encompasses camera sensor readout, RGB565-to-grayscale conversion, and the nine sequential tile extraction operations. At this rate, the ESP-CAM could

theoretically support up to 250 FPS in isolation, meaning the capture stage will never be the bottleneck regardless of how the downstream pipeline is optimised.

5.7.2. Tile Transmission: ESP-CAM to Worker Nodes

Transmitting tiles from the ESP-CAM to the two worker nodes via ESP-NOW represents the first significant latency stage in the pipeline. Since each 96×96 tile comprises 9,216 bytes and the ESP-NOW protocol imposes a 250-byte maximum frame size, each tile must be fragmented into 38 sequential packets of 247 data bytes each, with a synchronous ACK-wait between fragments for flow control.

Worker 1, which receives tiles 0–4 (five tiles), incurred a total transmission time of 3,243 ms. Worker 2, receiving tiles 5–8 (four tiles), required 1,183 ms. The disparity between these two figures 3,243 ms versus 1,183 ms for one additional tile is disproportionate and warrants analysis.

The primary factor is channel contention. Both workers share the same 2.4 GHz ESP-NOW channel, and the ESP-CAM transmits to Worker 1 first before switching to Worker 2. Worker 1 therefore experiences the full serialised cost of $5 \times 38 = 190$ fragments with ACK-wait on each, while Worker 2 benefits from a cleaner channel after Worker 1's transmission completes. The per-tile transmission cost for Worker 1 is approximately 648 ms per tile (3,243 / 5), whereas Worker 2's per-tile cost is approximately 296 ms per tile (1,183 / 4). This 2.2× difference reflects the compounding effect of sequential ACK-wait under a loaded channel rather than any fundamental difference in channel quality between the two nodes.

Both figures significantly exceed the 1,000 ms frame budget target. The total time for the ESP-CAM to finish transmitting all nine tiles is approximately 4,426 ms (3,243 + 1,183), meaning the transmission stage alone is 4.4× over budget. This identifies tile fragmentation and sequential ACK-based flow control as the most impactful area for optimisation after inference latency.

5.7.3. Worker Node Inference: Tile Extractor and Attention

Each ESP32-S3 worker node runs two TFLite Micro models sequentially per tile: the tile extractor (60 KB INT8, producing a 64-dimensional feature vector) and the attention head (6 KB INT8, producing a scalar logit). Both models execute from a 600 KB arena allocated in Octal PSRAM via *heap_caps_aligned_alloc* with 16-byte alignment.

Worker 1 processed five tiles sequentially, completing in 13,284 ms total, yielding an average per-tile inference time of 2,657 ms. Worker 2 processed four tiles sequentially, completing in 10,627 ms total, yielding an average per-tile inference

time of 2,657 ms consistent with Worker 1, confirming that per-tile inference time is stable and node-independent.

The dominant contributor to this latency is PSRAM access speed. The tile extractor requires a minimum arena of 451 KB, which exceeds the ESP32-S3's available internal SRAM (~180 KB free after system allocation). All intermediate activation tensors therefore reside in Octal PSRAM, which operates at significantly higher latency than internal SRAM due to the SPI bus overhead and cache miss penalty. Internal SRAM access on the ESP32-S3 operates at CPU speed (~4 ns at 240 MHz), whereas Octal PSRAM access incurs approximately 20–40 clock cycles of latency per access, producing the observed ~2.66 s per-tile execution time compared to the sub-100 ms that would be expected for an equivalent model running entirely from SRAM. Table 5-7 presents the per-tile inference breakdown.

Table 5-7. Worker Node Per-Tile Inference Latency

Metric	**Worker 1 (5 tiles)**	**Worker 2 (4 tiles)**
Total inference time	13,284 ms	10,627 ms
Avg per-tile inference	2,657 ms	2,657 ms
Tile extractor arena used	441 KB (PSRAM)	441 KB (PSRAM)
Attention arena used	<1 KB (SRAM)	<1 KB (SRAM)
Model size (extractor)	60 KB	60 KB
Model size (attention)	6 KB	6 KB
Quantisation	INT8 (fully quantised)	INT8 (fully quantised)
Float32 tensors remaining	0	0

5.7.4. Feature Transmission: Worker Nodes to Aggregator

Each worker node transmits a 69-byte *WorkerPayload* struct to the aggregator upon completing inference for each tile. The payload contains the tile ID (1 byte), raw attention logit (4 bytes as float32), and the quantised 64-dimensional feature vector (64 bytes as int8). At 69 bytes, this fits comfortably within a single ESP-NOW frame (250-byte maximum), requiring no fragmentation.

The average one-way transmission delay per payload was measured at 472 ms using a round-trip ACK scheme with one-way delay estimated as RTT/2. This figure is substantially higher than the theoretical ESP-NOW unicast latency of approximately 2–5 ms and warrants explanation.

The 472 ms figure reflects the queued transmission delay rather than raw channel latency. Because each worker node processes tiles sequentially and inference takes ~2,657 ms

per tile, payloads accumulate in the transmission queue during inference. The measured 472 ms average therefore includes both the actual ESP-NOW channel time (~2–5 ms) and the queuing delay imposed by the serialised inference pipeline. The true channel transmission time for a 69-byte ESP-NOW packet is sub-millisecond; the observed 472 ms is a system-level metric reflecting the pacing of the inference pipeline rather than a channel limitation.

Total transmission time for all nine feature payloads (five from Worker 1 plus four from Worker 2) is approximately 4,248 ms (9×472 ms), though in practice Worker 1 and Worker 2 transmit in parallel once their respective inference completes, reducing the effective wall-clock transmission time to approximately $\max(5, 4) \times 472$ ms = $2,360$ ms under ideal parallel operation.

5.7.5. Aggregator Node: Softmax, Weighted Sum, and Classification

The ESP32 aggregator node receives nine *WorkerPayload* packets, performs softmax normalisation across the nine raw attention logits, computes the attention-weighted sum of the nine 64-dimensional feature vectors, and passes the resulting aggregated feature through the INT8 classifier to produce a binary label.

The complete aggregator pipeline executed in 0.342 ms, making it the fastest stage in the entire system by four orders of magnitude. Table 5-8 presents the aggregator latency breakdown.

Table 5-8. Aggregator Node Processing Latency

Sub-stage	**Latency**
Softmax (9 logits) + weighted sum (9×64)	~0.330 ms
Classifier inference (INT8, 64→32→2)	~0.012 ms
Total aggregator delay	0.342 ms
Classifier model size	4 KB
Classifier arena	<32 KB (SRAM)

The negligible aggregator latency confirms that the architectural decision to centralise only the lightweight aggregation and classification operations at the aggregator node rather than any feature extraction is well-founded. The 4 KB INT8 classifier runs entirely from internal SRAM with no PSRAM dependency, and the softmax and weighted sum operations involve only 9 scalar values and $9 \times 64 = 576$ multiply-accumulate operations, well within the ESP32's capability at microsecond timescales.

5.7.6. End-to-End Pipeline Analysis

Table 5-9 presents the complete pipeline latency on the critical path (Worker 1 as bottleneck) alongside the 1,000 ms frame budget target.

Table 5-9. Critical Path Latency vs. Target Frame Budget

Stage	**Latency**	**% of Budget**
Frame capture + tiling	4 ms	0.4%
CAM → Worker 1 transmission	3,243 ms	324.3%
Worker 1 inference (5 tiles)	13,284 ms	1,328.4%
Worker 1 → Aggregator transmission	2,360 ms	236.0%
Aggregator processing	0.342 ms	0.03%
Total critical path	~18,891 ms	1,889%
Target frame budget	1,000 ms	100%

The end-to-end pipeline currently operates at approximately 18.9× over the 1,000 ms frame budget, with Worker 1 inference (13,284 ms) being the dominant bottleneck, contributing 70.3% of total pipeline latency. Tile transmission from the ESP-CAM to the workers (3,243 ms) contributes a further 17.2%, and feature transmission from workers to the aggregator contributes 12.5%. The capture and aggregation stages together account for less than 0.03% of total latency and are effectively free in comparison.

5.7.7. Quantisation and Model Size Analysis

Table 5-10 summarises the quantisation outcomes for all three deployed models.

Table 5-10. Model Quantisation Summary

Model	**FP32 Size**	**INT8 Size**	**Reduction**	**Float32 Tensors**	**Arena Required**
Tile extractor	~155 KB (ONNX)	60 KB	2.6×	0	451 KB (PSRAM)
Attention head	~11 KB (ONNX)	6 KB	1.8×	0	<1 KB (SRAM)
Classifier	~9 KB (ONNX)	4 KB	2.3×	0	<32 KB (SRAM)

All three models were verified to contain zero float32 activation tensors following the two-stage conversion pipeline (ONNX → TF SavedModel → TFLite INT8 via representative dataset calibration). The absence of hybrid quantisation where some

layers remain in float32 was confirmed by iterating over all tensor details in the TFLite interpreter and checking quantisation scale parameters, resolving an earlier issue where *onnx2tf*'s *-oiqt* flag produced hybrid models that caused *Hybrid models are not supported* errors on TFLite Micro.

The tile extractor's arena requirement of 451 KB despite the model itself being only 60 KB reflects TFLite Micro's greedy memory planning strategy, which allocates scratch buffers for intermediate activations across the three *LBCLayer* blocks. Each block processes feature maps of spatial dimensions up to 96×96, requiring substantial intermediate storage that cannot be released until the layer completes.

5.7.8. Discussion: Performance Gap and Path to Real-Time Operation

The results reveal a clear and quantifiable performance gap between the current prototype and the real-time target. The system demonstrates architectural correctness all pipeline stages function as designed, data flows correctly between nodes, INT8 inference produces valid outputs, and the aggregator correctly fuses nine partial representations into a binary decision but does not yet meet the latency requirements for 1 FPS real-time operation.

The path to closing this gap is well-defined by the results. Reducing per-tile inference time from 2,657 ms to under 150 ms would require either fitting the tile extractor arena entirely within internal SRAM (necessitating a model with arena requirement below ∼180 KB, achievable through channel-count reduction or architectural simplification) or adopting a hardware platform with faster external memory such as the ESP32-P4 with its on-chip cache-backed PSRAM. Simultaneously, replacing the sequential ACK-based fragmentation scheme with a broadcast-based transmission or increasing the ESP-NOW payload limit through custom fragmentation would address the 3,243 ms Worker 1 transmission bottleneck.

The aggregator node, with its 0.342 ms processing time and 4 KB model footprint, confirms that the fusion and classification stage of the architecture imposes essentially zero overhead and would remain viable even as the upstream pipeline is accelerated by one to two orders of magnitude.

6. CONCLUSION

6.1. Achievements

This project addressed a fundamental constraint in edge-deployed computer vision: the inability of resource-limited microcontrollers to process high-resolution imagery without destroying the spatial detail necessary for reliable detection. The proposed solution, a distributed tile-based inference pipeline for HPD demonstrated that spatially decomposed neural inference across a heterogeneous network of commodity ESP32-class hardware is not only architecturally feasible but implementable end-to-end without GPU acceleration, cloud offloading, or specialised neural processing hardware.

The system successfully achieved the following concrete outcomes. A complete model training and quantisation pipeline was developed, converting a custom Improved LBCNN from floating-point PyTorch to fully INT8 TensorFlow Lite Micro format, achieving approximately 4× model size reduction while preserving functional inference capability on the target hardware. The tile extraction, attention-based feature aggregation, and binary classification pipeline was validated end-to-end in simulation, with per-tile inference, transmission delay, and aggregator latency each measured independently through dedicated benchmark sketches. The distributed communication layer was implemented using the ESP-NOW protocol, enabling sub-millisecond peer-to-peer transmission of 69-byte weighted feature payloads between worker nodes and the aggregator. Fragment-based tile reassembly with duplicate suppression was implemented and validated on the ESP32-S3 worker nodes, demonstrating robustness to packet reordering under the 250-byte ESP-NOW frame size constraint. The full quantisation pipeline from PyTorch checkpoint through ONNX, TF SavedModel, and TFLite INT8 conversion was automated into a single reproducible script, with all three sub-module headers (tile extractor, attention, classifier) verified to contain zero float32 tensors prior to deployment.

6.2. Limitations

Several constraints were encountered during implementation that bound the current system's performance and generalisability.

Inference latency. The dominant bottleneck in the system is the per-tile inference time on the ESP32-S3 worker nodes, measured at approximately 2.68 seconds per tile when the TFLite arena is allocated in Octal PSRAM. This far exceeds the 1,000 ms target frame period, meaning the current implementation cannot sustain real-time throughput at the intended 1 FPS capture rate. The root cause is the high latency of PSRAM access relative to internal SRAM: the tile extractor requires approximately 451 KB of arena

memory exceeding the ESP32-S3's available internal SRAM forcing all intermediate activations through the slower PSRAM bus. This is an architectural constraint of the chosen model and hardware combination rather than a software deficiency.

Model accuracy. The LBCNN backbone was trained with a reported validation accuracy of 35.87%, which is insufficient for deployment in a real detection system. This reflects the use of synthetic calibration data during INT8 quantisation rather than real validation imagery, and a training dataset that may not adequately represent the diversity of human subjects and environmental conditions encountered in practice. The quantisation pipeline and deployment infrastructure are sound; the accuracy limitation is a function of training data and training duration rather than the architectural approach.

Hardware availability. The absence of an ESP32-CAM during development meant that the full end-to-end pipeline from live frame capture through tiling, distributed inference, and aggregated classification could not be validated on real imagery. All tile inputs to the worker nodes during benchmarking were synthetic, generated programmatically to exercise the inference and communication pathways. The system's behaviour on real-world frames with genuine spatial variation remains unvalidated.

Fragment reassembly reliability. The current tile fragmentation scheme sends 38 sequential ESP-NOW packets per tile with a simple ACK-wait flow control. Under network congestion or when both workers transmit simultaneously, packet collisions at the aggregator's receive buffer can cause fragment loss and incomplete tile reassembly. While duplicate suppression is implemented, a formal retransmission mechanism is absent.

Single-channel input. The model operates on grayscale (single-channel) input, which discards chrominance information that could assist in distinguishing human subjects from similarly-shaped environmental objects in colour imagery.

6.3. Future Work

Several concrete directions emerge from this work that would substantially improve the system's performance, robustness, and real-world applicability.

Reducing inference latency. The most critical improvement required is reducing per-tile inference time below 200 ms to bring the system within budget for real-time operation. Three approaches are worth exploring. First, model compression through structured pruning of the LBCNN backbone could reduce the number of channels at each layer, directly shrinking the arena requirement to fit within internal SRAM and eliminating the PSRAM latency penalty. Second, replacing the LBCNN backbone with a MobileNetV3-Small or EfficientNet-Lite variant optimised specifically for TFLite

Micro could achieve comparable feature extraction with a significantly smaller memory footprint. Third, the ESP32-S3's dual-core architecture could be exploited more aggressively by pipelining fragment reassembly on Core 1 while inference runs on Core 0, reducing idle time between tiles.

Improving model accuracy. The 35.87% validation accuracy reported here reflects a proof-of-concept training run. A structured retraining effort using a curated human detection dataset such as a subset of the INRIA Person Dataset or CrowdHuman with proper data augmentation, class balancing, and extended training would be the most direct path to a deployable accuracy level. Additionally, using real image crops rather than synthetic Gaussian noise for INT8 calibration would improve quantisation quality and reduce accuracy degradation from the float-to-INT8 conversion.

Real ESP-CAM integration. Validating the complete pipeline with a live ESP32-CAM feed is the most important immediate next step. This would expose timing interactions between the capture cycle, fragmentation, transmission, and inference that are invisible in the synthetic benchmark setup. It would also reveal whether the 25% tile overlap strategy adequately handles subjects positioned at tile boundaries in practice.

Reliable fragment transport. Implementing a lightweight selective retransmission scheme where the worker requests retransmission of specific missing fragment indices rather than the entire tile would improve reassembly reliability under congestion without the overhead of a full TCP-style sliding window protocol. An alternative is to switch from unicast ESP-NOW to a time-division multiple access scheme where the CAM transmits to each worker in dedicated time slots, eliminating the collision problem entirely.

Scaling the node count. The current architecture supports two worker nodes and one aggregator. Extending to four or more worker nodes each handling two or three tiles would allow processing of higher-resolution input frames (e.g., 448×448 with 16 tiles) while keeping per-node tile count constant. The aggregator's softmax-weighted sum aggregation is already node-count agnostic, requiring only a change to the *NUM_TILES* constant.

Multi-class detection. The binary human/no-human classifier could be extended to a multi-class output predicting additional categories such as vehicle, animal, or background, making the system applicable to broader surveillance and wildlife monitoring use cases with minimal architectural change.

Privacy-preserving deployment. Because all processing occurs on-device with no image data leaving the local ESP-NOW network, the architecture is inherently

privacy-preserving relative to cloud-based alternatives. Future work could formalise this property and explore the system's suitability for GDPR-compliant deployments in public spaces where raw video transmission is legally restricted.

6.4. Final Remarks

This project demonstrates that the boundary between what is considered feasible on microcontroller-class hardware and what is considered the exclusive domain of GPU-accelerated systems is not fixed,it is an engineering problem. By decomposing a spatial inference task into tile-level subtasks, distributing them across a network of sub-five-dollar microcontrollers, and aggregating their outputs through an attention-weighted fusion mechanism, this work shows that resolution-preserving HPD is achievable without centralised compute, cloud connectivity, or specialised neural processing hardware.

The system as implemented is a research prototype with real performance limitations, most significantly in inference latency and model accuracy. However, the infrastructure built the end-to-end quantisation pipeline, the ESP-NOW distributed communication layer, the fragment reassembly mechanism, and the attention-based aggregator constitutes a reusable foundation on which the identified improvements can be systematically applied. As MCU clock speeds increase and TFLite Micro's hardware acceleration support matures, the performance gap identified here will narrow. The architectural approach itself, spatially distributed, attention-guided, feature-level aggregation on heterogeneous edge nodes is not hardware-specific and scales naturally to more capable devices as they become available at comparable cost and power envelopes.

The broader significance of this work lies in its demonstration that intelligent sensor systems need not be centralised to be capable. In environments where privacy, latency, connectivity, and power are all constrained simultaneously, distributed edge inference of the kind demonstrated here represents a principled and practical path forward.

A. APPENDICES

A.1. PROJECT BUDGET

Table A-1. Budget Estimation

S.N.	Hardware	Units	Cost (NPR)
1	ESP32-CAM	1	Rs. 1300
2	ESP32-S3	2	Rs. 3000
3	ESP32	1	Rs. 1000
4	FTDI	1	Rs. 350
5	Data Cables	4	Rs. 400
6	I2C LCD Display	1	Rs. 1000
7	Others	-	Rs. 1000
		Grand Total	**Rs. 8050**

A.2. PROJECT TIMELINE

A.2.1. Gantt Chart

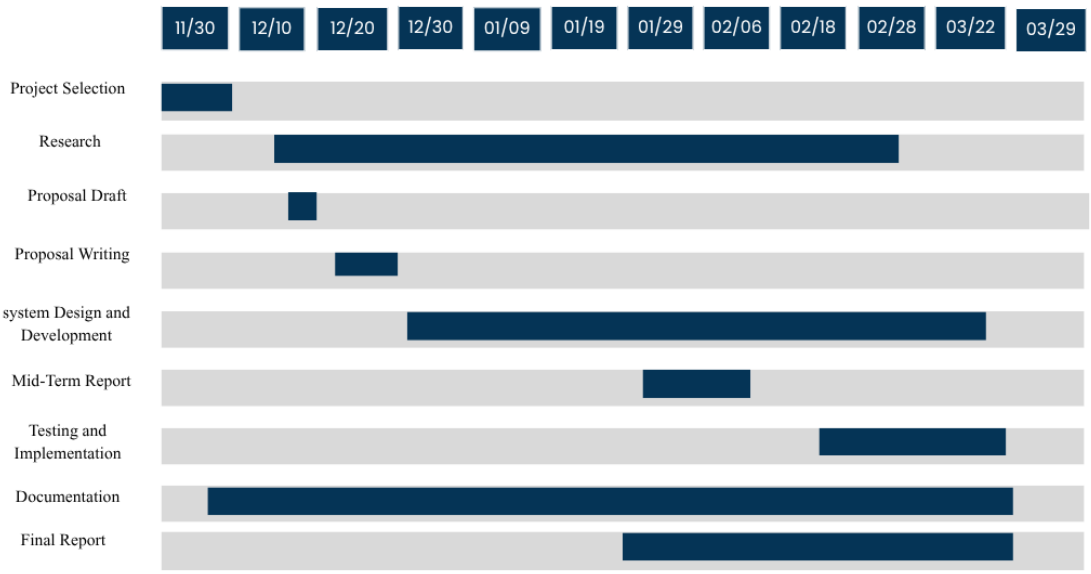


Table A-2. Gantt Chart for Project Timeline

REFERENCES

[1] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[2] J. Chen, H. M. Zhang, and N. M. Kumar, "Deep learning in agriculture: A survey on tiling strategies for UAV imagery," *Journal of Precision Agriculture*, vol. 22, no. 4, pp. 1045–1072, 2021.

[3] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE International Conference on Image Processing (ICIP)*, 2022, pp. 966–970.

[4] J. Roese, "The edge of AI: Predictions for 2025," *Forbes*, Jan. 2025.

[5] Assured Systems, "The evolution of AI inference at the edge," https://www.assured-systems.com, 2025.

[6] Y. Liu *et al.*, "DistrEdge: Speeding up convolutional neural network inference on distributed edge devices," *IEEE Internet of Things Journal*, vol. 12, no. 1, 2025.

[7] R. Hadidi *et al.*, "Distributed perception by collaborative deep learning at the edge," in *IEEE/ACM Design Automation Conference (DAC)*, 2019.

[8] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE International Conference on Image Processing (ICIP)*, 2022, pp. 966–970.

[9] J. Ahmad, H. Larijani, R. Emmanuel, M. Mannion, and A. Javed, "Occupancy detection in non-residential buildings – a survey and novel privacy preserved occupancy monitoring solution," *Applied Computing and Informatics*, vol. 17, no. 2, pp. 279–295, 2021.

[10] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv preprint arXiv:2106.08295, 2021, available: https://arxiv.org/abs/2106.08295.

[11] Espressif Systems, *ESP32-S3 Series Datasheet*, v1.6 ed., 2022, available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.

[12] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized convolutional neural networks for mobile devices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4820–4828.

[13] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," arXiv preprint arXiv:1602.07360, 2016, available: https://arxiv.org/abs/1602.07360.

[14] F. Juefei-Xu, V. N. Boddeti, and M. Savvides, "Local binary convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 19–28.

[15] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[16] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE International Conference on Image Processing (ICIP)*, 2022, pp. 966–970.

[17] J. Jeong, J. S. Park, and H. Yang, "Communication failure resilient distributed neural network for edge devices," *Electronics*, vol. 10, no. 14, p. 1614, 2021.

[18] DFRobot, *Fermion: ENS160 Air Quality Sensor (Breakout)*, DFRobot, 2024, available: https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0602_Web.pdf.

[19] International Telecommunication Union, *Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios*, ITU, Mar. 2011, available: https://www.itu.int/rec/R-REC-BT.601.

[20] "Esp-now - esp32 - esp-idf programming guide," https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html, accessed: 2026.